MODULE 12

Lab 2: Build a Mini Assistant Loop

Duration	Format	Audience	Day / Position
45 min	**Hands-On Lab**	**Mixed Engineers**	**Day 2 / Fourth**

LAB OVERVIEW

In this hands-on lab, you will combine everything from Modules 9–11 into a complete agent-assisted development workflow. Starting with the sensor toolkit from Lab 1, you'll observe the codebase, create a subagent, propose a change via plan mode, implement it with an approval gate, run quality checks, and draft a PR — all in 45 minutes.

This is a guided lab. Follow each exercise in order. The instructor is available for help but won't demo each step. If you get stuck, use /rewind to recover or ask for help.

- Follow exercises sequentially
- Use /rewind if stuck

PREREQUISITES

- Completed Lab 1 (Module 6) sensor toolkit project with passing tests
- Git repository initialized with at least one commit
- CLAUDE.md configured with TI coding standards
- Familiarity with subagents (Module 10) and plan mode (Module 11)

LAB TIMING

0:00 – 0:07	**Exercise 1: Repository Observation & Context Summary**
0:07 – 0:15	**Exercise 2: Create Your First Production Subagent**
0:15 – 0:25	**Exercise 3: Plan a Feature with Plan Mode**
0:25 – 0:35	**Exercise 4: Implement, Test & Quality Gate**
0:35 – 0:42	**Exercise 5: Draft PR & Code Review**
0:42 – 0:45	**Checkpoint: Verify, Commit & Debrief**

0:00 – 0:07	EXERCISE 1: Repository Observation & Context Summary

Exercise 1: Repository Observation & Context Summary

Before making any changes, have Claude analyze the project you built in Lab 1 to establish a baseline understanding.

Step 1: Navigate to your Lab 1 project

```
cd ~/my-sensor-toolkit
# Verify git status:
git status
git log --oneline -5  # Should show Lab 1 commits
```

Step 2: Ask Claude to observe the repository

```
# Open Claude Code:
claude

# Ask for a full repository analysis:
> Analyze this repository and create a comprehensive summary.
> Include: project structure, tech stack, module purposes,
> test organization, coverage status, and CLAUDE.md rules.
> Output the summary to docs/repo-summary.md
```

Step 3: Review the summary

```
# Read what Claude generated:
> Show me docs/repo-summary.md

# Verify it captured key details:
# - Project: sensor-toolkit
# - Modules: validators.py, cleaners.py, analyzers.py
# - Tests: conftest.py with fixtures, test_validators.py, etc.
# - Tools: pytest, ruff, black, coverage
# - CLAUDE.md: Google style, type hints, line length 100
```

Step 4: Identify a feature to add

```
# Ask Claude for improvement suggestions:
> Based on your analysis, suggest 3 features that would
> improve this sensor toolkit. For each, estimate:
> - Complexity (Low/Medium/High)
> - Files affected
> - Time to implement with Claude Code
```

Track A (IT Engineers)	Track B (Process Engineers)
Choose: Add a REST API endpoint for sensor validation (uses Flask/FastAPI)	Choose: Add a CSV export function for cleaned sensor data

More complex: multiple new files, dependencies, integration tests	Focused: one new function in analyzers.py, unit tests

✅ EXERCISE 1 CHECKPOINT

- docs/repo-summary.md exists and is accurate
- You've identified which feature to implement
- You know which files will be affected

0:07 – 0:15	EXERCISE 2: Create Your First Production Subagent

Exercise 2: Create Your First Production Subagent

Create a code reviewer subagent tailored to your sensor toolkit project.

Step 1: Create the agents directory

```
mkdir -p .claude/agents
```

Step 2: Use /agents to create a subagent

```
# In Claude Code, use the interactive agent creator:
> /agents

# Select: Create new agent
# Select: Project-level (.claude/agents/)
# Select: Generate with Claude

# Describe the subagent:
> A sensor toolkit code reviewer that checks Python code
> against TI coding standards: Google style, type hints,
> docstrings, validation range correctness for sensor data
> (temp: -40..150, pressure: 0..1000, humidity: 0..100).
> Read-only access. Output prioritized findings.

# Press Enter to generate, then e to edit if needed
```

Step 3: Verify and customize the subagent

```
# Check the generated file:
> cat .claude/agents/sensor-reviewer.md

# Verify YAML frontmatter:
# - name: sensor-reviewer
# - tools: Read, Grep, Glob, Bash  (or Read-only subset)
# - model: inherit

# Customize if needed (e.g., restrict to read-only):
> Edit the sensor-reviewer subagent to use only
> Read, Grep, and Glob tools. No Write or Bash.
```

Step 4: Test the subagent

```
# Invoke the subagent explicitly:
> Use the sensor-reviewer subagent to review
> @src/sensor_toolkit/validators.py

# Claude delegates to the subagent, which:
# 1. Reads the file
# 2. Checks against TI standards
# 3. Returns prioritized findings
# 4. Returns to main context
```

☆ **KEY POINT:** You just created a reusable, shareable code reviewer that any team member can use. Commit .claude/agents/sensor-reviewer.md to git and it's available to everyone on the project.

✅ **EXERCISE 2 CHECKPOINT**

- .claude/agents/sensor-reviewer.md exists with valid YAML
- Tools are restricted (no Write access for reviewer)
- Subagent successfully reviewed validators.py
- Findings are organized by priority (Critical/Warning/Suggestion)

0:15 – 0:25	EXERCISE 3: Plan a Feature with Plan Mode

Exercise 3: Plan a Feature with Plan Mode

Use plan mode to design your feature before writing any code. This creates an explicit approval gate.

Step 1: Activate plan mode

```
# Toggle plan mode on:
> Shift+Tab
# You should see [PLAN MODE] indicator
```

Step 2: Describe the feature (Track A)

```
# Track A: REST API endpoint
> [PLAN MODE] Add a /api/validate endpoint to the sensor
> toolkit using FastAPI. The endpoint should:
> - Accept POST requests with JSON sensor readings
> - Validate using existing validate_reading() function
> - Return validation results with HTTP 200 (valid) or
>   422 (validation errors) with error details
> - Include OpenAPI documentation
> Plan the implementation. Do NOT write code yet.
```

Step 2: Describe the feature (Track B)

```
# Track B: CSV Export function
> [PLAN MODE] Add a csv_export() function to analyzers.py
> that takes a list of cleaned SensorReadings and writes
> them to a CSV file with columns: timestamp, sensor_id,
> temperature, pressure, humidity, status.
> Include: header row, proper datetime formatting,
> pathlib.Path for file paths, type hints.
> Plan the implementation. Do NOT write code yet.
```

Step 3: Review and refine the plan

```
# Claude produces a plan. Review it:
> # Typical plan output:
> # 1. Files to create/modify:
> #    - src/sensor_toolkit/analyzers.py (add csv_export)
> #    - tests/test_analyzers.py (add csv_export tests)
> #    - requirements-dev.txt (no changes needed)
> # 2. Implementation details: ...
> # 3. Test plan: happy path, empty list, special chars
> # 4. Edge cases: ...

# Refine if needed:
> Add to the plan: also handle sensor_id values with
> special characters by quoting CSV fields properly.
> And add a parameter for custom column ordering.
```

Step 4: Save the plan

```
# Ask Claude to save the plan:
> Save this plan to docs/feature-plan.md

# Turn off plan mode:
> Shift+Tab  # Back to normal mode
```

☆ **KEY POINT:** The plan is your contract. When you execute it, Claude follows it exactly. If you skip planning, Claude makes its own decisions about architecture and approach — which may not match your team's expectations.

✅ **EXERCISE 3 CHECKPOINT**

- docs/feature-plan.md exists with a detailed implementation plan
- Plan includes: files affected, implementation steps, test plan
- You've reviewed and refined the plan
- Plan mode is now off (back to normal mode)

0:25 – 0:35	EXERCISE 4: Implement, Test & Quality Gate

Exercise 4: Implement, Test & Quality Gate

Execute the plan from Exercise 3, run tests, and verify quality metrics.

Step 1: Commit current state (safety checkpoint)

```
# Create a checkpoint before implementing:
> !git add -A && git commit -m "pre-feature: baseline"

# Now you have a safe point to /rewind to
```

Step 2: Implement the feature

```
# Tell Claude to execute the plan:
> Implement the feature described in @docs/feature-plan.md.
> Follow the plan exactly. Use existing patterns from
> validators.py and cleaners.py as reference.

# Claude implements step by step:
# - Creates/modifies files per the plan
# - Follows CLAUDE.md conventions
# - Uses existing fixture patterns for tests
```

Step 3: Run tests

```
# Run the full test suite:
> !pytest tests/ -v --cov=src/sensor_toolkit \
>   --cov-report=term-missing

# If tests fail:
> These tests failed: [paste failures].
> Fix them while preserving the existing passing tests.

# If fix doesn't work:
> /rewind
> # Try a different approach, or ask Claude to
> # implement a simpler version
```

Step 4: Run the quality gate

```
# Lint check:
> !ruff check src/ tests/

# Format check:
> !black --check src/ tests/

# If lint/format issues exist:
> Fix all ruff violations and run black formatter.

# Diff size check:
> !git diff --stat
```

```
# Should be < 200 lines for a single feature

# Coverage check:
> !pytest --cov=src/sensor_toolkit --cov-report=term-missing
# New code should have >= 80% coverage
```

If Quality Gate Passes	If Quality Gate Fails
Continue to Exercise 5	/rewind to the pre-feature commit
All tests pass, lint clean, coverage ≥ 80%	Ask Claude to fix specific issues
Diff is reasonable size (<200 lines)	If multiple failures: /rewind and simplify the feature scope

✅ EXERCISE 4 CHECKPOINT

- Feature code is implemented following the plan
- All tests pass (existing + new)
- Ruff lint: zero violations
- Black format: clean
- Coverage: new code has ≥ 80% coverage
- Diff is < 200 lines

0:35 – 0:42	EXERCISE 5: Draft PR & Code Review

Exercise 5: Draft PR & Code Review

Generate a professional PR description and run your subagent code review.

Step 1: Generate PR description

```
# Ask Claude to draft the PR:
> Generate a GitHub PR description for the changes since
> the "pre-feature: baseline" commit. Include:
> - Title (conventional commit format: feat: ...)
> - Summary of changes
> - Files modified with brief explanations
> - Testing: what tests were added and coverage
> - Breaking changes (if any)
> Save to docs/pr-description.md
```

Step 2: Run subagent code review

```
# Use the subagent you created in Exercise 2:
> Use the sensor-reviewer subagent to review all
> changes since the "pre-feature: baseline" commit.

# The reviewer subagent will:
# - Read the diff
# - Check against TI standards
# - Verify validation ranges
# - Report findings by priority
```

Step 3: Address review findings

```
# If the reviewer found Critical issues:
> Fix the critical issues identified by the reviewer:
> [paste the critical findings]

# Re-run tests after fixes:
> !pytest tests/ -v

# Re-run the reviewer to verify:
> Use the sensor-reviewer subagent to verify the fixes.
```

Step 4: Final commit

```
# Generate a commit message:
> Generate a conventional commit message for these changes.

# Commit everything:
> !git add -A
> !git commit -m "feat: add csv_export with tests and docs"

# Tag the milestone:
> !git tag v0.2.0 -m "Lab 2: mini assistant loop complete"
```

☆ **KEY POINT:** You just completed a full agent-assisted development cycle: observe → plan → approve → implement → test → quality gate → review → commit. This is the same workflow you'll use in production, just faster with practice.

0:42 – 0:45	CHECKPOINT: Verify, Commit & Debrief

Final Checkpoint & Debrief

FINAL VERIFICATION

✅ LAB 2 COMPLETE CHECKLIST

- docs/repo-summary.md — repository observation document
- .claude/agents/sensor-reviewer.md — production subagent with restricted tools
- docs/feature-plan.md — approved implementation plan
- New feature implemented and passing all tests
- Quality gate passed: lint clean, format clean, coverage ≥ 80%
- docs/pr-description.md — professional PR description
- Subagent code review completed with findings addressed
- Final commit with conventional message and v0.2.0 tag

WHAT YOU BUILT IN 45 MINUTES

Step	Manual Time	With Claude Code
Repository analysis & documentation	30–60 min	3 min
Create & configure a code review agent	N/A (not possible manually)	5 min
Feature planning with impact analysis	30–45 min	5 min
Implementation with tests	2–4 hours	10 min
Quality gate (lint, format, coverage)	15–30 min	2 min
PR description + code review	30–45 min	5 min
TOTAL	4–7 hours	30 min

THE ASSISTANT LOOP PATTERN

What you just practiced is the core agent-assisted development loop that scales to any project:

1. OBSERVE: Understand the codebase before changing it (repo summary, file localization, CLAUDE.md)
2. PLAN: Use plan mode to design the change with human approval gate
3. IMPLEMENT: Execute the approved plan with subagents, following existing patterns
4. VERIFY: Run quality gates (tests, lint, format, coverage, diff size)
5. REVIEW: Use reviewer subagent for automated first-pass code review
6. SHIP: Generate PR description, commit with conventional message, tag release

DEBRIEF QUESTIONS

- *Which step of the assistant loop added the most value for you?*
- *What would you change about your subagent after using it?*
- *Where in your daily TI workflow would this loop save the most time?*
- *What guardrails would your team need before using this in production?*